Writing skills *for Claude Code* agents.

Skill features and templates for AI agents

A practical authoring guide, extracted from a working QA agent setup

Canache Gabriel
QA Engineer
Centric Romania

Version 1.0 · October 2026

centric

Contents

1. Why I wrote this **4**

A quick word on agents and skills . . . 4

2. What lives in the automation folder **5**

3. The building blocks of a skill **7**

The description . . . 7
Where it sits in the flow . . . 7
A short context block . . . 7
A table of references . . . 8
Templates for everything it writes . . . 8
Scripts and helpers . . . 8
Simple commands as input . . . 9
Being specific about tools . . . 9
Where the truth comes from . . . 9
A numbered workflow . . . 10
Tracking the run on the board . . . 10
Checking the work . . . 10
A limit on retries . . . 11
Ending with a clear report . . . 11
A human decides . . . 11
Ids that never change . . . 12
Safe to run twice . . . 12
Guardrails . . . 12
Lessons . . . 13
One real example . . . 13

4. Putting it together **14**

The skill . . . 14
The agent . . . 15
A reference file . . . 15
An output file waiting for review . . . 16

5. Commands I use every day **17**

Adding an MCP server . . . 17
Calling an agent . . . 18

Permissions 18

6. Before you run a new skill **20**

1. Why I wrote this

A few notes I wish I had when I started writing skills for our QA agents.

I built five Claude Code agents for our QA work. One reads a user story and writes a test plan, one explores the running application and writes down what it finds, two of them turn the approved findings into test cases and bugs in Azure DevOps, and the last one automates the test cases in Playwright.

An agent without a clear structure tends to drift. It reads files it does not need, changes things nobody asked it to change, or fills a gap with a guess. What keeps mine predictable is that every skill follows the same structure.

This document is that structure. I took my five skills, looked at what they have in common, and wrote each common piece down with a short explanation and a template you can copy. Nothing here is specific to our application. You should be able to use it for any agent, QA or not.

A quick word on agents and skills

In Claude Code these are two different files, and the difference matters.

The **agent** file (`.claude/agents/<name>.md`) decides what the agent is allowed to use: which tools, which model, how permissions behave. It runs in its own context and gives you back a report when it is done. The **skill** (`.claude/skills/<name>/SKILL.md`) is the procedure: what the agent should do, step by step, and what it must never do.

I keep them as pairs with the same name. The agent file stays short and strict, and the skill holds all the explanation. If you only remember one thing from this page, let it be this: the agent file is the lock, the skill is the instructions on the door.

2. What lives in the automation folder

Everything the agents need is in one place, so you always start Claude Code from here.

This is the content of my `automation` folder. The part that matters for this guide is `.claude`, but the other folders explain where the agents read from and where they write to.

automation/

```
.claude/
├── agents/                          # one file per agent: tools, model, skill
│   ├── qa-planner.md
│   ├── qa-explorer.md
│   ├── qa-test-case-writer.md
│   ├── qa-bug-reporter.md
│   └── qa-automator.md
└── skills/                          # one folder per skill
    ├── qa-planner/
    │   ├── SKILL.md                 # the entry point
    │   ├── references/              # loaded only when a step needs them
    │   │   ├── ado-qa-task.md
    │   │   ├── example.md
    │   │   ├── feature-map.md
    │   │   └── test-data-constraints.md
    │   └── scripts/
    │       └── check-feature-map.mjs
    ├── qa-explorer/
    │   ├── SKILL.md
    │   └── references/  (ado-qa-task, environment, fixtures, output-templates)
    ├── qa-test-case-writer/
    │   ├── SKILL.md
    │   └── references/  (ado-test-case-writing, proposed-test-case-format,
    │                     test-case-template)
    ├── qa-bug-reporter/
    │   ├── SKILL.md
    │   └── references/  (ado-bug-reporting, bug-file-format, bug-template, severity)
    └── qa-automator/
        ├── SKILL.md
        └── references/  (ado-automation, framework, test-data-and-cleanup)
.mcp.json                            # MCP servers shared by all agents
.gitignore                           # keeps archive/ and .secrets/ out of git
lessons/                             # one lessons file per agent
└── qa-<agent>-lessons.md
tools/
└── qa-login.mjs                     # signs in without showing credentials
archive/<WORK_ITEM_ID>/              # agent outputs and hand-offs (git-ignored)
├── QA_PLAN.md
├── TEST_SCENARIOS.md
└── bugs/BUG-NN-<slug>.md
.secrets/                            # local credentials (git-ignored)
fe_automation/                       # Playwright + TypeScript project
├── baseTest/                        # fixtures, global setup, login
├── pageObjects/<portal>/            # one class per page
├── tests/<portal>/*.spec.ts         # specs
├── testData/                        # data factories
├── db/queries/*.sql                 # setup and cleanup queries
└── playwright.config.ts
```

A few things are worth pointing out. The agent files are tiny: just a block of settings at the top. All the real content is in the skills, and each skill keeps its `SKILL.md` short by moving details into `references/`.

The `archive` folder is where the agents talk to each other. The planner writes a plan there, the explorer reads it and writes its results next to it, and so on. I keep it out of git on purpose, because it is working material that a human reviews, not something to ship.

`lessons` is the agents' memory. Every time I correct an agent, the correction goes into its lessons file, and the agent reads it at the start of the next run.

> **One practical thing**
> Claude Code looks for agents, skills and `.mcp.json` in the folder you start it from. If the agents don't show up when you type `@`, you are probably in the wrong folder.

3. The building blocks of a skill

These are the pieces I ended up repeating in every skill. Not every skill needs all of them, but most need most of them.

I grouped them the way I think about them when I write a new skill: first who the agent is, then what it knows, what it can touch, how it works, how it hands work over, and finally how it stays safe and learns.

Who the agent is

The description

The `description` in the frontmatter is the most important sentence in the skill. Claude uses it to decide when to pick the skill, and it is also the first thing a colleague reads when they open the file.

I try to answer four questions in it: what does it read, what does it produce, what is the one rule that matters most, and where can I see that it ran. It ends up being a long sentence, and that is fine.

```
---
name: <skill-name>
description: Reads <what> from <where>, produces <what> in <where>,
  <the most important rule>, and tracks the run in <task or file>
  (created if missing, set in progress, summary written, then closed).
---
```

Where it sits in the flow

Right after the title I write one paragraph that says which step comes before this one and which comes after. Just as important, I say what the agent does *not* do.

My explorer, for example, finds bugs but never files them; that is the bug reporter's job. Writing it down keeps the explorer from creating work items on its own.

```
# <Skill Title>

The <Skill Title> is the step after `<previous-skill>`. It <main job>.
It does not <the thing another agent does>; that is `<other-skill>`'s job.
```

A short context block

Next, a few lines about the world the agent works in: the application, the URLs, the tracker. Keep it short.

I also tell it what it already has. Our `CLAUDE.md` is loaded automatically, so every skill says “do not read it again”. For a very large document I tell it how to read it: search for the heading first, then read only that part. It saves a surprising amount of time and tokens.

```
**System under test:** <app> - API <url>, UI <url>, database <name>.
**Tracker:** <organisation> / <project>.
`CLAUDE.md` is already in your context - do not Read it again.
For <large file>: grep the `## ` heading, then Read only that section.
```

What the agent knows

A table of references

Long skills are hard to follow, for the model and for people. What works better is keeping `SKILL.md` short (somewhere between 120 and 200 lines) and moving the details into separate files under `references/`.

The trick is the small table that says *when* to read each file. The agent then loads the bug template only when it is about to write a bug, not at the start. Skills can also share references: three of mine use the same feature map.

```
## References (load only when the step needs it)

| File                                  | When                                |
|---------------------------------------|-------------------------------------|
| [references/<board>.md](...)          | Steps 0 and N - the tracking task |
| [references/<format>.md](...)         | Step 1 - how to read the input    |
| [references/<template>.md](...)       | Step 4 - the output template      |
| [../<other-skill>/references/<x>.md] | Step 2 - shared with <other-skill>|
```

Templates for everything it writes

Anything the agent produces, whether a plan, a bug or a test case, gets a template in `references/`. Same headings, same field names, every time.

This made reviewing much faster for me, and it made the next agent in the chain reliable, because it always knows where to find the steps or the expected result.

```
# references/<output>-template.md

## Title
<Where>: <what goes wrong>          (max 80 characters)

## Steps
1. <action>

## Expected / Actual
- Expected: <result> (source: <requirement>)
- Actual:   <result>

## Evidence
<screenshot path, response excerpt, correlation id>
```

Scripts and helpers

Some checks should not depend on the model reading carefully. If something can be checked by code, I put a small script in `scripts/` with clear exit codes. My planner has one that checks whether every document it points to still exists.

The other kind of helper is about secrets. Agents need to log in to the application, but I never want a password in their context. So a small script in `tools/` reads the credentials, logs in, and prints only “OK” or an error.

```
// scripts/check-<thing>.mjs
// Checks that <every entry in X still exists in Y>.
// Exit 0 = all good, 1 = stale entries (listed), 2 = could not read the inputs.

// tools/<login>.mjs
// Reads credentials from .secrets/<file>.env and signs in.
// Prints a one-line result. Never the password, never the token.
```

What the agent can touch

Simple commands as input

All my agents take the same kind of command: a verb and an id. `Plan Story 112`, `Report bugs 112`, `Automate PBI#50`. On top of that, two variants that turned out to be really useful.

A list of ids limits the run to a few items, which is handy when only one bug was approved. And “dry run” makes the agent do everything except write, so I can see what it would create before it touches the board.

```
## Inputs
- `<Verb> <ID>`            - every eligible item of that work item.
- `<Verb> <ID> <id> <id>`  - only those items.
- `<Verb> <ID> dry run`    - read and plan only, no writes.
- A Task id works too: use its parent and say so.
- Never ask for credentials; sign in through the helper.
```

Being specific about tools

The agent file says which tools exist. The skill should say what each tool is for. For every tool I write what it may read, what it may write, and then “nothing else”.

I also say what to do when a tool fails. Without that, an agent that loses its Azure DevOps connection may look for another way in, for example through the browser. With the rule in place, it just asks me to reconnect.

```
## Tools / MCP servers
- `<tracker>` - read <items>. Write ONLY <one item type or field>.
  Nothing else. Auth error -> ask the human to reconnect (/mcp).
- `<database>` - read-only SELECT.
- `Bash` - only `<command>`.
- `Write` / `Edit` - only inside <folder> and the lessons file.
- If `<required server>` is not available, stop and say which one.
```

Where the truth comes from

This one is very QA-specific, but I think it applies more widely. My planner and explorer are not allowed to look at the source code, pull requests or commits. Expected results come only from the requirements, the API contract and the documentation.

Otherwise the agent reads the code, sees what it does, and calls that the expected behaviour. Which is exactly the bias a tester is supposed to avoid. I also tell it which source wins when two of them disagree.

```
**Black-box.** Expected results come only from <requirements>, <API contract>
and <docs>. Never open source code, tests, pull requests or commits.
If the acceptance criteria and a document disagree, the acceptance criteria
win; list the difference as a gap. If only the code could answer a question,
put it under Open Questions.
```

How the agent works

A numbered workflow

The procedure is always a numbered list, and it always starts at 0. Step 0 is housekeeping: read the lessons, find the work item, mark the tracking task as in progress.

Each step after that does one thing and says when to stop. “No test cases found, say so and stop” sounds obvious, but without it the agent will happily invent something to do.

```
## Workflow
0. Read the lessons file. Resolve the work item, find or create the
   tracking task, set it in progress (references/<board>.md).
1. Collect <inputs>. Nothing found: say so and stop.
2. Read <sources> (references/<x>.md).
3. <main action>.
4. Verify the result.
5. Report to the human, write the summary, close the task.
```

Tracking the run on the board

Every agent run shows up on the board as a small task under the user story: “QA Plan”, “QA Exploration”, “QA Automation”. The agent finds it or creates it, moves it to in progress, writes a summary at the end and closes it.

This sounds like overhead, but it is the reason the rest of the team trusts the agents. They can see what ran and what it found without asking me.

```
## Tracking task (references/<board>.md)
1. Read the work item and find the child Task whose title starts with
   `<prefix>`. If there are several, use the most recent one.
2. None? Create it under the work item, same area and iteration.
3. Set it to in progress.
4. At the end, write the summary in its Description and add a comment
   like `<n> passed / <n> failed / <n> blocked`.
5. Close it only if everything succeeded. Otherwise leave it open and say why.
```

Checking the work

I learned to ask for proof, not just completion. Before an agent reports anything, it re-reads its output against the rules. My automator also has to show that the tests really ran (the right number of them, none skipped) and that no test data was left in the database afterwards.

One check I like a lot: change the expected value on purpose and make sure the test fails. A test that cannot fail does not test anything.

```
## Verify
- Re-read every output against the Guardrails before sending it.
- The tests really ran: <count> found, none skipped.
- The assertion fails when the expected value is changed on purpose.
- No data left behind: rows with the run tag = 0.
```

A limit on retries

When a test fails, the automator gets three attempts to fix it, and it may only change how the test runs (a locator, a wait, the data setup), never what it checks.

Without this rule, the easiest way for an agent to turn a test green is to loosen the assertion until it passes. The test goes green and the bug stays. With the rule, if the failure looks like a real defect, it stops and reports it.

```
## When a test fails
- At most 3 fix attempts, each followed by a re-run.
- Change only HOW the test runs, never WHAT it checks.
- If it looks like a real bug: stop, leave it failing, report the evidence.
- After the third attempt, leave it as it is and move on.
```

Ending with a clear report

The last step is always a report to me, in the same shape every time: what was created, what was skipped and why, and what needs my attention. A small table works best. I want to read it in ten seconds and know what to do next.

```
## Report
| Item | Result | Created id | Notes |
|------|--------|------------|-------|
Then: already done (with id), skipped (with status),
invalid (with what to fix), blocked (with the action I need to take).
```

How agents hand work over

A human decides

Nothing goes to Azure DevOps without a person looking at it first. The explorer writes every bug as a file with a status line set to `PENDING`. I read it and change it to `CONFIRMED` or `REJECTED`.

The bug reporter and the test case writer only act on confirmed items. This single line is what makes it safe to let agents write to the tracker at all.

```
## Reviewer Decision
Status: PENDING        <!-- I write CONFIRMED / ACCEPTED or REJECTED -->

# and in the next agent's guardrails:
- Act only on CONFIRMED or ACCEPTED items.
- Never on PENDING, REJECTED or anything unclear. Never change a decision.
```

Ids that never change

Every scenario gets an id like `FE1` or `BE3`, and every bug gets `BUG-01`. Once written, an id is never renumbered. The next agent matches items by id, not by reading the text and guessing.

The plan also ends with a short “Handoff” section for the next agent: what to log in to, what to check first, and in which order to run things.

```
FE1. [<role>] <scenario> - <expected result> - <source>
     - `automate: yes|no · regression: yes|no · testcase: yes|no`

### Handoff
- Sign in to: <portals / roles>
- Check first: <unverified preconditions>
- Order: <shared-data scenarios together, state-consuming ones last>
```

Safe to run twice

Sooner or later you run the same command twice. The agents that create things in Azure DevOps first check whether the item already exists, and after creating it they write the new id back into the file.

The second run sees the id and skips the item. No duplicates, no cleanup.

```
1. Skip items that already have `<id field>:` in the file.
2. Search the tracker for <unique tag> under the work item.
   Found: don't create it, write its id back, report "already done".
3. Create the item.
4. Write back `<id field>: <new id>` and `<date field>: <date>`.
```

Staying safe and getting better

Guardrails

Every skill ends with a short list of rules that win over everything else in the file. I keep them few and blunt.

The ones I repeat everywhere: the agent may change only one kind of thing, it never touches credentials, it uses only test data it created, it never invents facts, and it never edits the application or commits anything.

```
## Guardrails
- The only thing you may create or change is <...>. Everything else is read-only.
- Never read <secrets folder> or <.env>, never print a password or token.
- Use only synthetic, run-tagged data (@example.test). Never touch data
  you did not create.
- Every fact comes from the source. If something is missing, say so.
- Never edit <application folders>, never reset the environment,
  never commit or push.
```

Lessons

Each agent has a lessons file. It reads it at the start of every run, and when I correct it, it adds the correction there.

Two rules keep the file useful: a lesson never overrides a guardrail, and the agent only adds lessons when I tell it something, never things it concluded on its own.

```
# lessons/<skill>-lessons.md

## YYYY-MM-DD - <short title>
- Lesson: <the rule>
- Why: <what went wrong, or what I said>
- How to apply: <when and how it changes the work>
```

One real example

I finish every skill with an example run: the command, what the agent finds, what it creates and what it reports. One concrete example explains more than a page of rules, both to the model and to the next person who reads the skill.

```
## Example
User: "<Verb> 123."

The agent finds <n> items: <a> confirmed, <b> rejected. It reads work item 123,
finds no duplicates, creates <a> items linked to it, writes their ids back,
and reports: <a> created, <b> skipped.
```

4. Putting it together

The same pieces, assembled into files you can copy and fill in.

The skill

.claude/skills/<skill-name>/SKILL.md

```
---
name: <skill-name>
description: Reads <what>, produces <what>, <key rule>, and tracks the run in
  <task> (created if missing, set in progress, summary written, then closed).
---

# <Skill Title>

The <Skill Title> is the step after `<previous>`. It <main job>. It does not <...>.

**System under test:** <urls>. **Tracker:** <org / project>.
CLAUDE.md is already in your context - do not Read it again.

## References (load only when the step needs it)
| File                | When        |
|---------------------|-------------|
| references/<a>.md   | Steps 0, N  |
| references/<b>.md   | Step 3      |

## Inputs
- `<Verb> <ID>` - <default scope>
- `<Verb> <ID> <ids>` - only those items
- `<Verb> <ID> dry run` - read and plan only

## Tools / MCP servers
- `<server>` - read <...>. Write ONLY <...>. Nothing else.
- `Bash` - only `<command>`.
- `Write` / `Edit` - only <folder> and the lessons file.

## Workflow
0. Read lessons, resolve the item, set the task in progress.
1. Collect <inputs>. Nothing found: say so and stop.
2. <step> (references/<b>.md)
3. Verify: re-read against the Guardrails; <proof>.
4. Report, write the summary, close the task.

## Guardrails
- The only thing you may create or change is <...>.
- No credentials, synthetic data only, never edit the app, never commit.

## Lessons
Read `lessons/<skill>-lessons.md` first. Add to it only when a human corrects you.

## Example
User: "<Verb> 123." -> <what it finds, writes and reports>
```

The agent

The agent file is short. Most of the thinking goes into the `tools` list: give it only what the skill really uses. If you leave `tools` out completely, the agent gets every tool you have, which is rarely what you want.

.claude/agents/<skill-name>.md

```
---
name: <skill-name>
description: <the same sentence as the skill description>
tools:
  - Read
  - Glob
  - Grep
  - Edit
  - Skill
  - mcp__<server>              # the whole server, or mcp__<server>__<tool>
model: opus
effort: high
permissionMode: acceptEdits     # file edits without asking
skills:
  - <skill-name>
---
```

A reference file

.claude/skills/<skill-name>/references/<topic>.md

```
# <Topic> in <tool>

<One paragraph: what this file covers, and the only thing the skill may change.>

Tools (`<server>` MCP):
- read:  `<tool>` (`get`, `get_batch`)
- write: `<tool>` (`create`, `update`)

## 1. Find the work item
## 2. Find or create <the item>
## 3. Write <the result>
## 4. Close and comment
```

An output file waiting for review

archive/<ID>/bugs/BUG-01-<slug>.md

```
---
work_item: <ID>
id: BUG-01
---
# BUG-01 - <Where>: <what goes wrong>

## Steps to reproduce
1. <action>

## Expected
<result> (source: <requirement>)

## Actual
<result>

## Evidence
<screenshot / response excerpt / correlation id>

## Reviewer Decision
Status: PENDING
```

5. Commands I use every day

Adding an MCP server, calling an agent, and controlling what it is allowed to do.

Adding an MCP server

MCP servers are how the agents reach the outside world: Azure DevOps, the database, the API contract, the browser. There are two ways to add one.

The first is to edit `.mcp.json` directly. It sits next to `.claude` and is committed, so everyone on the team gets the same servers.

.mcp.json

```
{
  "mcpServers": {
    "<server-name>": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "<package>@latest", "<arg>"],
      "env": { "<TOKEN_NAME>": "${<ENV_VARIABLE>}" }
    },
    "<remote-server>": {
      "type": "http",
      "url": "https://<host>/mcp"
    }
  }
}
```

Never put a token in this file. The `${...}` syntax reads it from an environment variable instead. On Windows, `npx` needs to be wrapped: use `"command": "cmd"` with `"args": ["/c", "npx", ...]`.

The second way is the command line, which writes the same file for you:

```
# shared with the team (goes into .mcp.json)
claude mcp add --scope project <server-name> -- npx -y <package>@latest

# with an environment variable
claude mcp add --scope project <server-name> -e <TOKEN_NAME>=<value> -- <command> <args>

# a remote server over HTTP
claude mcp add --transport http <server-name> <https-url>

claude mcp list                # what is configured and whether it connects
claude mcp remove <server-name>
```

Without `--scope project` the server is only added for you, on your machine. Inside a session, `/mcp` shows which servers are connected and lets you log in again when a token expires. Once a server is connected, its tools are named `mcp__<server>__<tool>`, and that is the name you use in agent files and permission rules.

Calling an agent

Type `@` in the prompt, pick the agent from the list (they show up as `agent-<name>`), and write the task the way the skill expects it:

```
@agent-qa-planner Plan Story <ID>
@agent-qa-explorer Explore Story <ID>
@agent-qa-test-case-writer Write test cases <ID> dry run
@agent-qa-bug-reporter Report bugs <ID> BUG-01 BUG-03
@agent-qa-automator Automate PBI#<ID>
```

The agent works in its own context and comes back with a report. You can also just ask in plain words (“use the qa-planner agent to plan story 112”), but the `@` mention leaves no doubt about which agent runs. `/agents` is where you list and edit them, and `/<skill-name>` runs a skill directly in your current session.

Permissions

This is the part I am still improving. Right now most of my restrictions live in the skill text, and the skill text is only an instruction. The model follows it very well, but nothing forces it to.

There are two places where Claude Code actually enforces limits. The first is the `tools` list in the agent file. If `Write` or `Bash` is not there, the agent cannot use it. For MCP servers you can give the whole server or a single tool:

.claude/agents/<name>.md

```
tools:
  - Read
  - Edit                                # can edit, but no Write and no Bash
  - mcp__<server>__<read-tool>          # just one tool of the server
  - mcp__<server>__<write-tool>
permissionMode: acceptEdits             # or default, plan, bypassPermissions
```

The second is a `settings.json` file with allow, ask and deny rules. These apply to every agent, whatever its permission mode, so they are the right place for things that must never happen, like reading the secrets folder.

.claude/settings.json

```
{
  "permissions": {
    "allow": [
      "Bash(node tools/<helper>.mjs:*)",
      "Read(../docs/**)",
      "mcp__<server>__<read-tool>"
    ],
    "ask": [
      "mcp__<server>__<write-tool>"
    ],
    "deny": [
      "Read(./.secrets/**)",
      "Read(./**/.env)",
      "Edit(../<application-folder>/**)",
      "mcp__<server>__<dangerous-tool>"
    ]
  }
}
```

Deny always wins over ask, and ask wins over allow. `Read(...)` and `Edit(...)` take folder patterns, `Bash(...)` takes the start of a command, and MCP rules use the `mcp__<server>__<tool>` names. The team file is `.claude/settings.json`; for personal rules use `.claude/settings.local.json`, which stays out of git.

`/permissions` shows you what is currently in effect.

My advice: write the rule in the skill so the agent understands it, and add the same rule to the agent's tools or to `settings.json` so it is actually enforced.

6. Before you run a new skill

The questions I ask myself before I let a new skill loose.

1. Does the description say what it reads, what it produces and where I can see that it ran?
2. Does the first paragraph say where it sits and what it does not do?
3. Is `SKILL.md` short, with the details in references that are tied to a step?
4. Does every output have a template?
5. Are the inputs simple commands, with a dry run?
6. For every tool, is it clear what it may read and what it may write?
7. Is the workflow numbered, with clear places to stop and a check at the end?
8. Is there a limit on retries, and does the run end with a report?
9. Does anything leave the workspace without a human approving it first?
10. Are there guardrails, a lessons file and one real example?
11. Is what the skill promises also enforced in the agent's tools or in `settings.json`?

If the answer to all of these is yes, try it with a dry run first. It is the cheapest way to catch a mistake.

Canache Gabriel · QA Engineer · Centric Romania